

1. Atividades Práticas - Contador Avançado com useReducer

Nesta atividade prática, vamos utilizar o **useReducer** para gerenciar o estado de um contador, de maneira que o usuário possa incrementar, decrementar ou resetar o contador.

1.1 Criando o projeto e iniciando o servidor local

Crie um projeto chamado **compras**, digitando o seguinte comando no terminal:

```
npm create vite@latest contador-adv -- --template react
```

Este processo de configuração inicial do projeto leva alguns segundos. Ao terminar, o Vite repassa instruções para que você termine de instalar as dependências do seu projeto. Desta forma, digite o seguinte comando para **entrar no diretório** recém criado do nosso projeto:

```
cd contador-adv
```

Em seguida, **instale as dependências** necessárias do projeto executando no terminal o seguinte comando:

```
npm install
```

Até aqui, você criou um projeto React usando o Vite e adicionou todas as dependências ao projeto. Você pode, agora, **abrir a aplicação no editor Visual Studio Code**. Para isso, dentro do diretório do projeto digite o seguinte comando no terminal:

```
code .
```

Após a execução deste comando, o Visual Studio Code deverá abrir com a pasta raiz do seu projeto sendo acessada. Em seguida, você inicializará um servidor local e executará o projeto em seu navegador. Digite a seguinte linha de comando no terminal:

```
npm run dev
```

Ao executar esse script, você iniciará um servidor local de desenvolvimento, executará o código do projeto, iniciará um observador que detecta alterações no código e abrirá o projeto em um navegador web. Ele irá rodar a aplicação em modo desenvolvimento em <http://localhost:5173/>.

1.2 Passo 1: Criar a estrutura do projeto

Inicialmente, vamos criar a estrutura do nosso projeto. Neste contexto, dentro do diretório **src**, crie os seguintes diretórios:

- **components:** Contém os componentes visuais e de interação com o usuário.
- **reducers:** Contém a lógica do **useReducer**, separando a lógica de estado do componente.

1.3 Passo 2: Criar a Função Reducer

Vamos criar um **reducer** que recebe o estado atual e uma ação e decide como o estado deve ser atualizado com base no tipo da ação. Desta forma, dentro do diretório **reducers**, crie um arquivo chamado **counterReducer.jsx** com o seguinte código:

```
// Estado inicial do contador
const initialState = { count: 0 };

// Função reducer para gerenciar as ações do contador
function counterReducer(state, action) {
  switch (action.type) {
    case 'INCREMENT':
      return { count: state.count + 1 }; // Incrementa o valor
    case 'DECREMENT':
      return { count: state.count - 1 }; // Decrementa o valor
    case 'RESET':
      return { count: 0 }; // Reseta o valor
    default:
      return state;
  }
}
export { counterReducer, initialState };
```

Observe neste exemplo que:

- **initialState:** Define o valor inicial do contador como 0.
- **counterReducer:** Lida com três ações:
 - **INCREMENT:** Incrementa o valor do contador.
 - **DECREMENT:** Decrementa o valor do contador.
 - **RESET:** Reseta o valor do contador para 0.

1.4 Passo 3: Criar o Componente Principal

Agora, vamos implementar o componente React que usa o para gerenciar o estado do contador e exibir os botões de controle. Desta forma, dentro do diretório **components**, crie um arquivo chamado **Counter.jsx** com o seguinte código:

```
import { useReducer } from "react";
import { counterReducer, initialState } from "../reducers/counterReducer";

function Counter() {
  // Inicializa o useReducer com o reducer e o estado inicial
  const [state, dispatch] = useReducer(counterReducer, initialState);

  return (
    <div>
      <h2>Contador: {state.count}</h2>

      {/* Botões para incrementar, decrementar e resetar */}
      <button onClick={() => dispatch({ type: "INCREMENT" })}>
        Incrementar
      </button>
      <button onClick={() => dispatch({ type: "DECREMENT" })}>
        Decrementar
      </button>
      <button onClick={() => dispatch({ type: "RESET" })}>Resetar</button>
    </div>
  );
}

export default Counter;
```

Observe neste exemplo que:

- **useReducer(counterReducer, initialState):** Inicializa o **useReducer** com o **counterReducer** e o estado inicial (**{ count: 0 }**). Ele retorna o estado atual (**state**) e a função **dispatch** para disparar ações.
- **dispatch({ type: 'INCREMENT' }):** Dispara a ação **INCREMENT** para o **reducer**, que incrementa o valor do contador.
- **dispatch({ type: 'DECREMENT' }):** Dispara a ação **DECREMENT** para decrementar o valor do contador.
- **dispatch({ type: 'RESET' }):** Dispara a ação **RESET** para resetar o valor do contador para **0**.

1.5 Passo 4: Componente Principal

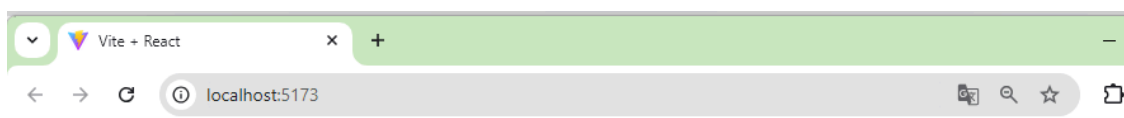
Finalmente, integramos o componente **Counter** ao componente principal **App**. Desta forma, altere o arquivo **App.jsx** com o seguinte código:

```
import Counter from './components/Counter';  
function App() {  
  return (  
    <div>  
      <h1>Exemplo de Contador com useReducer</h1>  
      <Counter />  
    </div>  
  );  
}  
export default App;
```

Neste exemplo, observe que o componente **App** renderiza o componente **Counter**, que gerencia o estado do contador usando o **useReducer**.

1.6 Passo 5: Testar a aplicação

Agora, execute o código, e você verá um contador que começa em 0.



Exemplo de Contador com useReducer

Contador: 0

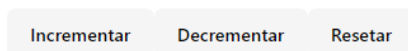


Figura 1: Exemplo da nossa aplicação sobre o contador

Em seguida, use os botões para incrementar, decrementar ou resetar o valor do contador. Observe como o estado do contador é gerenciado pelo **useReducer**, e as ações modificam o estado com base na lógica do reducer.